

Defesa - Explicação Técnica

Arthur Almeida De Souza & Hugo Gabriel Sales Soares

GitHub: [Repositório do Arthur](#)

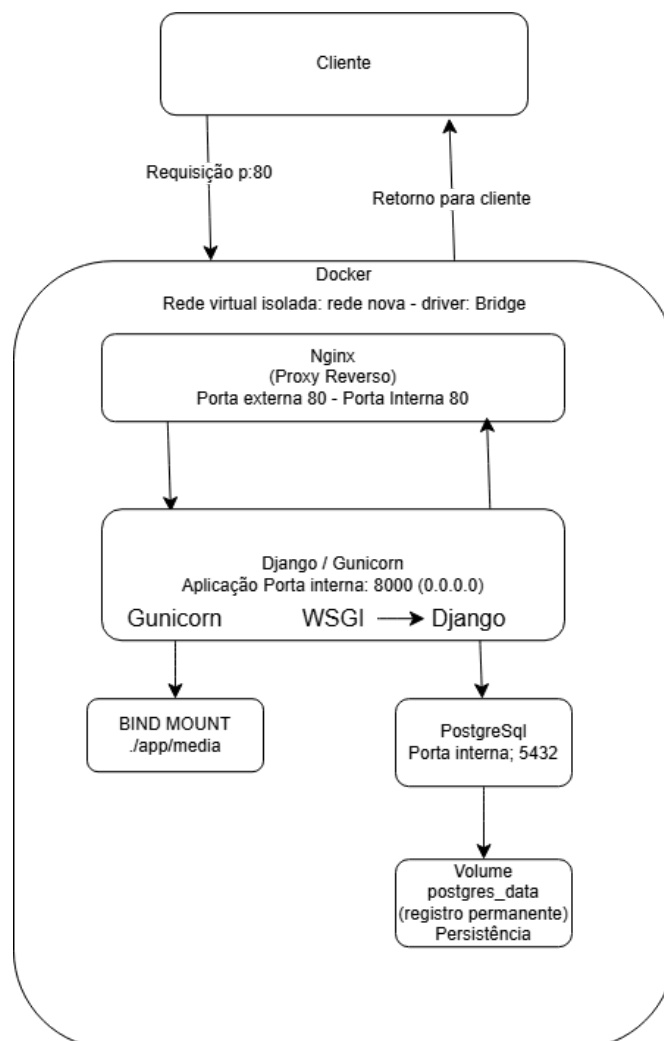
1. Visão geral da arquitetura

Infraestrutura (Docker Compose): Orquestra os serviços a partir do `docker-compose.yml`, conectando de forma segura através da rede privada `rede_nova`.

Proxy Reverso (Nginx): Funciona como escudo e única porta de entrada (porta 80), recebendo e repassando o tráfego externo para a aplicação.

Aplicação (Gunicorn, WSGI e Django): O Gunicorn recebe os pedidos na porta interna 8000. O WSGI traduz o HTTP para Python, permitindo que o Django processe a lógica. O código é espelhado via *bind mount* para atualizações em tempo real.

Banco de Dados e Persistência: O banco processa os registros dos formulários. *Bind mounts* salvam os uploads fisicamente na sua máquina, e *volumes nomeados* protegem as tabelas do banco, garantindo que nenhum dado seja perdido se a infraestrutura for desligada.



2. Imagem Docker da aplicação Django

A imagem escolhida para rodar nossa aplicação Django foi o **python:3.11-slim**. Foi escolhida, principalmente, a versão **slim**, por ser uma versão mais enxuta, que vem somente com o essencial para rodar o Python e não vai ficar muito pesada.

Depois, temos duas **ENVs** que dizem para não criar arquivos em byte code (.pyc), porque o Python, por padrão, cria esses arquivos .pyc quando executa módulos, e não armazenar saídas temporariamente no buffer, porque o Python também armazena algumas saídas antes de enviá-las para o terminal.

O diretório de trabalho (**WORKDIR**) que a imagem vai trabalhar é o **/app**. Se não existir, o Docker cria.

Depois, ele vai fazer um **COPY** do arquivo **requirements.txt**, que lista todas as dependências necessárias para que a aplicação funcione. Nele, contém o **Django**, **Gunicorn**, **psycpg2-binary** e o **Pillow**, e ele copia este arquivo para dentro da pasta **/app**.

Depois de copiar, ele roda o comando, durante o build da imagem, para instalar as dependências listadas no requirements e utiliza o **--no-cache-dir** para não armazenar arquivos temporários e economizar espaço na imagem.

Após isso, ele faz outro **COPY**, mas desta vez ele copia tudo o que está no ambiente do Dockerfile, todos os arquivos em volta dele, para a pasta dentro da pasta **/app** na imagem. Por fim, quando ele terminar de buildar e iniciar o container, ele vai rodar o **CMD**, que vai iniciar o **Gunicorn**, que é o servidor web que vai rodar a aplicação.

Depois, ele entra na pasta **django_docker_compose**, procura o arquivo **wsgi** e executa a função **application**. Aí, ele vai fazer um **--bind**, que diz ao Gunicorn qual IP e porta ele deve escutar.

Por fim, o **0.0.0.0:8000** é o IP e a porta que ele vai escutar/aceitar as requisições, onde ele vai aceitar as requisições vindas de qualquer IP e que a porta será a **8000**.

3. Orquestração com Docker Compose

O **docker-compose.yml** funciona como a **planta da infraestrutura do projeto**, centralizando a configuração e comunicação entre os containers.

- **db (PostgreSQL)**: responsável pelo armazenamento dos dados. Não expõe portas externamente, utiliza o volume **postgres_data** para manter os dados mesmo após a remoção do container e se conecta à **rede_nova**.
- **web (Django + Gunicorn)**: executa a aplicação, processa uploads e se comunica com o banco. É construída a partir do **Dockerfile** em **./app**, utiliza volumes para sincronizar código e arquivos enviados e depende do **db** para iniciar.

- **nginx (Proxy Reverso):** funciona como **porta de entrada do sistema**, recebendo requisições na porta 80 e encaminhando-as para o Django/Gunicorn. Também acessa os arquivos de configuração e uploads. Depende do serviço web.
- **Configurações globais:** a rede_nova permite a comunicação isolada entre os containers, enquanto os volumes garantem a persistência dos dados.

4. Comunicação entre os containers

A comunicação entre os containers ocorre por meio da **rede virtual rede_nova do Docker**, que possui DNS interno e permite que os serviços sejam encontrados pelo nome, sem necessidade de configurar IPs fixos.

- **Nginx → Django/Gunicorn:** o Nginx é o único serviço exposto externamente pela porta 80. Ele utiliza `proxy_pass http://web:8000` para encaminhar as requisições ao Django, usando o nome web para que o DNS interno do Docker encontre o container correto.
- **Django → PostgreSQL:** o banco (db) não possui portas expostas ao computador ou à internet. O Django utiliza `HOST='db'` para localizar o PostgreSQL através do DNS interno e estabelece a conexão pela porta 5432.

Em resumo: **usuário → Nginx:80 → Django/Gunicorn:8000 → PostgreSQL:5432**, com toda a comunicação interna protegida pela rede virtual do Docker.

5. Nginx e proxy reverso

O **Nginx atua como Proxy Reverso e única porta de entrada pública** da aplicação. Ele recebe as requisições na porta 80 e encaminha internamente para o Django/Gunicorn na porta 8000, que não é exposta diretamente.

A configuração do `nginx.conf` define esse comportamento:

- **listen 80:** recebe requisições HTTP na porta 80.
- **location /:** captura as requisições da aplicação.
- **proxy_pass http://web:8000:** encaminha as requisições para o serviço Django através da rede interna do Docker.
- **proxy_set_header:** preserva informações do cliente, como IP e host original, para que o Django possa utilizá-las.

- **location /media/**: permite que o Nginx entregue diretamente os arquivos enviados, como PDFs, sem precisar passar pelo Django.

Em resumo, o fluxo é: **cliente** → **Nginx:80** → **Django/Gunicorn:8000**. Para arquivos de mídia, o fluxo é **cliente** → **Nginx** → **diretório /media/**, reduzindo o processamento da aplicação.

6. Persistência de dados

Os containers Docker são **efêmeros**, ou seja, os dados armazenados diretamente neles podem ser perdidos quando são removidos. Por isso, a arquitetura utiliza duas formas de persistência:

- **PostgreSQL**: utiliza o volume nomeado `postgres_data`, que mantém os dados do banco mesmo após a remoção do container.
- **Arquivos enviados**: utiliza **Bind Mounts**, mapeando `./app/media` para os containers web e nginx. Assim, os PDFs ficam armazenados diretamente no disco da máquina.

No caso de executar `docker-compose down`, os containers são removidos, mas os dados permanecem. Ao executar `docker-compose up -d`, o Docker recria os containers e reconecta automaticamente o volume `postgres_data` e a pasta `./app/media`.

Em resumo: **containers podem ser recriados sem perda dos dados**, pois o banco e os arquivos possuem armazenamento independente do ciclo de vida dos containers.

7. Fluxo do upload de um arquivo

O upload segue um fluxo entre **Navegador** → **Nginx** → **Django/Gunicorn** → **armazenamento**, com o arquivo e seus metadados sendo persistidos separadamente.

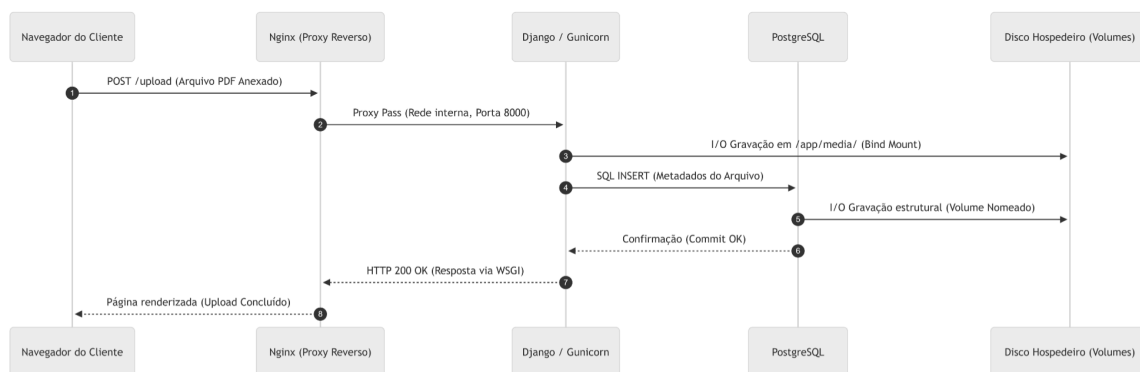
1. **Navegador**: o usuário seleciona o PDF e envia uma requisição POST usando `multipart/form-data` para a porta 80.
2. **Nginx**: recebe a requisição e a encaminha pela rede Docker para o Django/Gunicorn na porta 8000.
3. **Django/Gunicorn**: recebe o arquivo e processa a requisição através do WSGI.
4. **Armazenamento do arquivo**: o Django salva o PDF em `/app/media/`. Como esse diretório utiliza um **Bind Mount**, o arquivo é gravado diretamente na pasta `./app/media/` do computador.

5. **Registro no banco:** o Django envia um INSERT ao PostgreSQL, armazenando informações como nome, caminho e data do arquivo. Esses dados são persistidos no volume `postgres_data`.
6. **Resposta:** após o processamento, o Django retorna uma resposta de sucesso, que passa pelo Gunicorn e Nginx até chegar ao navegador.

Em resumo:

Navegador → Nginx → Django → PDF em `./app/media/` + metadados no PostgreSQL → resposta de sucesso.

O ponto importante é que **o arquivo físico e suas informações no banco são armazenados separadamente**, mas ambos permanecem disponíveis mesmo que os containers sejam recriados.



8.. Análise técnica da solução

Vantagens

- **Segurança e isolamento:** o PostgreSQL permanece restrito à rede interna, enquanto o Nginx centraliza o acesso externo.
- **Eficiência:** o Nginx entrega os PDFs diretamente, reduzindo o processamento do Django.
- **Portabilidade:** o `docker-compose.yml` permite recriar a mesma infraestrutura de forma padronizada em diferentes ambientes.

Limitações

- **Sem HTTPS:** o sistema utiliza apenas a porta 80, deixando o tráfego externo sem criptografia.
- **Bind Mount no código:** o `./app:./app` facilita o desenvolvimento, mas cria dependência do disco do servidor e não é ideal para produção.
- **Ponto único de falha:** existe apenas uma instância de cada serviço, portanto uma falha pode deixar o sistema indisponível.

Melhorias para produção

- **HTTPS:** utilizar Nginx na porta 443 com certificados SSL/TLS, como os fornecidos pelo Let's Encrypt.
- **Imagens imutáveis e armazenamento externo:** remover o Bind Mount do código e gerar uma imagem completa pelo Dockerfile. Para os PDFs, utilizar armazenamento de objetos, como AWS S3.
- **Segurança de credenciais:** retirar senhas do docker-compose.yml e utilizar .env não versionado ou Docker Secrets.

Em resumo, a arquitetura atual é **adequada para desenvolvimento e demonstração**, mas precisaria de **HTTPS, gerenciamento seguro de segredos, armazenamento mais robusto e mecanismos de alta disponibilidade** para um ambiente de produção.